```
15        fn conn ->
          conn
          |> Plug.Conn.put_resp_header("Content-Type", "application/json")
          |> Plug.Conn.resp(200, Jason.encode!(%{status: status}))
        end
20    )
  end

  test "a message status is updated" do
    message = SmsMessageFactory.create()
25  setup_bypass(message, status: "delivered")

    assert {:ok, job} = StatusWorker.enqueue(message)
    assert {:ok, updated} = StatusWorker.perform(job)

30  assert updated.status == "delivered"
    assert Repo.reload(message) == updated
  end

  test "not ready yet, enqueue" do
35  message = SmsMessageFactory.create()
    setup_bypass(message, status: "queued")

    assert {:ok, job} = StatusWorker.enqueue(message)
    assert StatusWorker.perform(job) == {:error, "Message not ready"}
40  assert Repo.reload(message) == message
  end
end
```

We use Bypass in our test to mock out several scenarios. It takes a few lines of code to set up Bypass, so the setup_bypass/2 function on line 7 handles this for us. Because of it, our test functions are easier to read.

Our test first enqueues a job (line 27) using the StatusWorker.enqueue/1 function. Then, the job is performed (line 28) and the expected result is asserted. Our test ensures that both enqueue/1 and perform/1 work. We repeat this process on line 38 to prove that the different logic paths in our worker are correct.

With that, we have a tested StatusWorker module! All of the different components of our application have tests too. There's a lot more that could be taught about testing, but this is a solid foundation for you to test your applications.

Wrapping Up

Tests are critical for production applications. They give you confidence that your application works as expected and will handle errors correctly. The way that you test doesn't matter as much as having the tests, so start early!